# bupty：纯 Bash 的北邮一站式信息查询终端

## Linux 课程作业——架构设计、实现与测试

2025211681

2026 年 10 月

# 目录

1. 动机
2. 功能
3. 架构
4. 设计选择
5. 测试
6. 总结

# 动机：信息分散在多个平台

- 北邮学生日常要用的信息，分散在好几个互不相干的系统里：
  - 查课表、空教室、成绩、考试——移动教务
  - 看作业与截止时间——教学云
  - 查班车、节假日、竞赛——公共信息接口
  - 预约图书馆研修间——另一个预约系统
- 每个系统都要单独打开、单独登录，来回切换很麻烦，还容易漏掉重要信息（比如作业截止、抢约时间）。

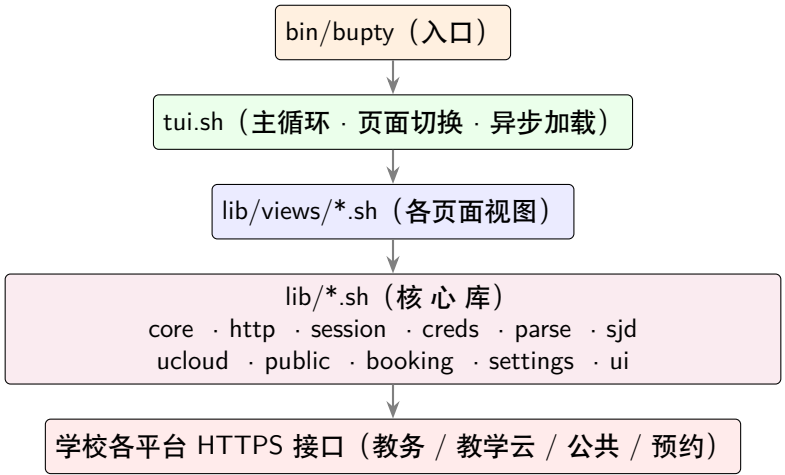
# 动机：把它们聚合起来，用 TUI 呈现

- **目标**：把上面这些信息聚合到「一个界面」里，减少来回切换。
- **为什么是 TUI（终端界面）而不是 CLI / GUI**：
  - 比 CLI（命令行）更友好：不用记一堆参数和命令，方向键 + Enter 就能操作。
  - 比 GUI（图形界面）更轻量：在终端里就能跑，无需图形环境。
- **为什么用 Bash**：课程要求；整个程序只用 Bash 加少量系统工具（curl 联网、jq 解析数据）实现，没有额外运行时依赖。

# 功能总览

- **作业 DDL**：教学云作业聚合、截止时间排序、紧急程度标注、在线提交。
- **课表**：周节次网格，可切周 / 切星期。
- **空教室**：双校区，按楼 / 楼层 / 时段筛选，可隐藏教室。
- **成绩 / 考试**：学期成绩、平均绩点、考试安排。
- **公共查询**：班车时刻、法定节假日、竞赛活动 DDL。
- **研修间预约**：浏览房间 / 空闲时段、预约、取消，以及「自动抢约」。
- **后台守护**：开机自启动 + 紧急 DDL 弹窗通知 + 抢约自动下单。
- **记住密码**：凭据本地安全保存，会话令牌仅存内存。

# 整体架构：分层


bin/bupty（入口）
tui.sh（主循环 · 页面切换 · 异步加载）
lib/views/*.sh（各页面视图）
lib/*.sh（核 心 库）
core · http · session · creds · parse · sjd
ucloud · public · booking · settings · ui
学校各平台 HTTPS 接口（教务 / 教学云 / 公共 / 预约）

# 每一层做什么（用大白话）

- **bin/bupty**：程序的「大门」。运行它，它负责找到代码所在位置，并把各个模块加载进来，然后启动界面。
- **tui.sh**：主循环。它一直在等用户按键，把按键交给「当前页面」处理；还负责页面的前进 / 返回（导航栈），以及后台加载数据后的自动刷新。
- **lib/views/*.sh**：一个个「页面」。每个页面只有两件事：`render_xxx` 画界面、`handle_xxx_key` 处理按键。
- **lib/*.sh**：被各页面共用的「工具箱」，例如网络请求、解析、保存密码、记住登录状态、画颜色等。
- **学校平台**：真正存放数据的地方。我们不是自己造数据，而是通过 HTTPS 接口去学校服务器「取数」。

# 核心库逐个看（lib/*.sh）

- core.sh：常量、日志、错误提示、颜色、文本宽度等最基础的工具（不联网）。
- http.sh：封装 curl，统一超时与请求头，返回状态码 + 响应体。
- session.sh：记住登录后拿到的「令牌」，只放内存、不落盘。
- creds.sh：读写账号密码文件（目录 0700、文件 0600，拒绝软链接）。
- parse.sh：纯解析函数（周次展开、节次解析、楼名归一），不联网、方便测试。
- sjd.sh：移动教务——登录、课表、空教室、成绩、考试。
- ucloud.sh：教学云——统一认证、课程列表、作业。
- public.sh：公共接口——班车、节假日、竞赛活动。
- booking.sh：研修间预约——登录、房间与空闲时段、预约、取消、抢约。
- settings.sh：读写设置文件（原子写，避免写一半损坏）。
- background.sh：后台守护——紧急 DDL 弹窗 + 自动抢约。
- submit.sh：作业在线提交。
- ui.sh：界面原语——清屏、边框、颜色、列表、输入框、按键读取。

# 页面视图逐个看（lib/views/*.sh）

- `home.sh`：首页菜单。
- `assignments.sh`：作业 DDL 页面。
- `schedule.sh`：课表页面。
- `classrooms.sh`：空教室页面。
- `grades.sh`：成绩页面。
- `exams.sh`：考试页面。
- `public.sh`：公共查询页面（班车 / 节假日 / 活动）。
- `booking.sh`：研修间预约页面。
- `settings.sh`：设置页面。

# 数据如何流动：JSON 一条道走到底

- **JSON 是一种通用的「文本格式」，用键值对表示数据，例如** `{"name":"张三","age":20}`。
- **我们规定：各模块之间传数据，一律用 JSON 字符串，读写都用命令行工具** `jq` **完成。**
- **好处：模块之间只约定「字段名」，不约定「分隔符」，不会因为数据里碰巧出现分隔符而出错。**
- **页面显示前，先用** `jq` **把数据做一次「预计算」整理成一行行的文本；真正逐行画界面时只用 Bash 本身，不再反复调用** `jq`**，因此更快。**
- **链路：网络模块** $\xrightarrow{\text{JSON}}$ **页面预计算** $\xrightarrow{\text{一行行文本}}$ **渲染。**

# 异步加载：界面始终不「卡住」

- 传统做法：进入页面 → 联网取数据 → 数据到了才显示。网络慢时，界面会空白或卡住。
- 我们的做法：进入页面后 **立刻**画出界面框架和「正在加载…」提示，数据在后台悄悄下载，下载完成后自动刷新。
- 实现上：用后台子进程去联网（`_async_start`），主循环每隔一小段时间检查一次是否完成（`_async_poll`），完成就更新页面。
- 渲染用「帧缓冲」：整屏内容先画到临时文件里，再一次性输出到终端，避免一行一行刷出来导致的闪烁。

# 渲染性能：去掉多余的「子进程」

- Bash 里很多写法（比如 $(命令)、管道）都会额外启动一个「子进程」，每启动一次都要花一点时间。
- 如果一个页面有几十上百行，每行都启动几次子进程，界面就会明显变慢。
- 我们的优化：
  - 渲染循环里改用 Bash 内置的运算，尽量不启动子进程；
  - 计算文字显示宽度用「一遍扫描」的算法（$O(n)$），避免反复裁剪导致的 $O(n^2)$ 卡顿；
  - 凭据、活动过滤、班车显示行等只算一次并缓存。
- 效果：课表、班车等页面从约 200ms 一帧降到约 10–30ms 一帧。

# 设计选择：把 Bash 也写得「模块化」

- 不把所有代码塞进一个超大脚本，而是按职责拆成几十个小文件，每个文件只负责一件事。
- Bash 里用 `source` 把其他文件「引入」进来，效果相当于把这些文件拼接在一起，但代码更清晰、更好维护。
- 命名约定：纯解析函数（`lib/parse.sh`）**不联网**，便于离线单元测试；网络函数成功返回 0、失败返回 1，结果写到标准输出、错误写到一个全局变量。

# 设计选择：安全

- 保存密码的目录权限设为 0700（只有本人能进）、文件 0600（只有本人能读写），写入时拒绝软链接，密码绝不打印、不写日志。
- 登录后服务器发的「令牌」（相当于一张通行证）只存在内存里，不写进磁盘；过期后自动重新登录一次。
- 预约系统使用学校的「统一身份认证」(CAS)：在统一登录页登录一次，就能访问多个系统；认证过程全程在同一个会话里顺序完成，一次性参数用后即弃。

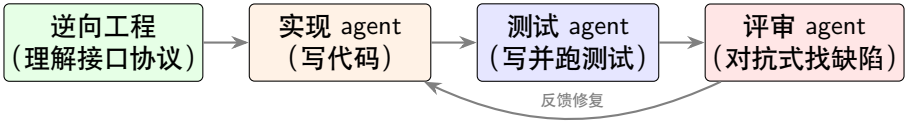
# 设计选择：可配置与后台

- 设置持久化到 `settings.json`，采用「先写临时文件、再原子替换」的方式，避免写一半损坏。
- `--background` 后台模式：只运行一份（单实例锁），循环拉取作业，对「紧急 DDL」弹窗提醒。
- 自动抢约只抢「明天及以后」的时段；遇到永久性错误（如已被禁用）就放弃并通知，临时错误则限次重试。

# 测试方法

- **单元测试**（`tests/unit_*.sh`）：离线、基于预置样例数据，断言解析 / 组装 / 过滤等纯函数的结果。
- **渲染快照**：渲染各页面并断言关键文字（去掉颜色码），验证界面文案正确。
- **多窗口尺寸自检**：在 80/100/120/60/140 列宽下断言不溢出、标题在、无横向换行。
- **真实网络集成测试**（只读 + 授权）：登录、拉取、预约 → 取消，验证协议与实现一致。
- **对抗式评审**：实现 / 测试 / 评审由不同 agent 分别完成，不共享上下文。

# 多代理开发流程




每一轮：一个 agent 实现、一个写测试、一个对抗式评审，彼此不共用上下文，避免「自我蒙混」。

# 总结

- 用纯 Bash 实现了一个功能完整、交互友好的北邮信息查询 **TUI**，把分散的信息聚合到一个界面。
- 核心经验：模块化拆分、JSON 作为数据契约、异步加载 + 帧缓冲、渲染热循环去子进程、O($n$) 宽度计算。
- 通过「单元测试 + 渲染快照 + 多尺寸自检 + 真实网络集成 + 对抗式评审」保证了正确性与界面质量。
- 各系统的网络接口均经真实请求逐条核对并文档化。

谢谢！